02 / GUÍA DE APRENDIZAJE

Guía de referencia Python

EX28 · NumPy: arrays 2D, transposición y productos

> **Orden de estudio:** 01 Teoría matemática e ingeniería → 02 Referencia Python → 03 Ejercicio → 04 Pseudocódigo → programación. Esta guía forma parte de EX28, clase 18.

1. Recordatorio: importación y construcción

Importamos `numpy` con el alias `np`. Un array 2D se construye escribiendo una lista de filas, y cada fila es otra lista. Todas las filas deben tener el mismo número de elementos.

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]], dtype=float)
x = np.array([2, -1], dtype=float)
```

La propiedad `dtype=float` fuerza valores reales de coma flotante. No es obligatoria para este ejercicio, pero resulta habitual en cálculo científico.

2. Propiedades de dimensiones

Expresión	Qué devuelve	Ejemplo
A.shape	Tupla (filas, columnas)	(2, 2)
A.ndim	Número de ejes	2
A.size	Cantidad total de elementos	4
x.shape	Forma de vector 1D	(2,)

Estas propiedades **no** son funciones: se consultan sin paréntesis, por ejemplo `A.shape`. Un array 1D de forma (2,) no es una matriz 2 × 1, aunque pueda utilizarse como vector en operaciones con @.

3. Operaciones componente a componente

```
B = np.array([[2, 0],
              [-1, 3]], dtype=float)

S = A + B        # Addition: same shape required
D = A - B        # Subtraction
K = 2.0 * A      # Scalar multiplication
H = A * B        # Element-wise multiplication
```

Entrada: dos arrays de formas iguales (para los ejemplos de suma, resta y producto elemento a elemento). Salida: nuevo array con la misma forma. NumPy admite reglas de broadcasting, pero no las utilizaremos en este ejercicio.

4. Transposición de una matriz

```
AT = A.T

print(A.shape)   # (2, 2)
print(AT.shape)  # (2, 2)
```

`A.T` devuelve una vista traspuesta para un array ordinario. También existe `np.transpose(A)`. En una matriz 2 × 3, la traspuesta tiene forma 3 × 2.

5. Producto matricial: operador @

```
y = A @ x        # Matrix (2,2) @ vector (2,) -> (2,)
C = A @ B        # Matrix (2,2) @ matrix (2,2) -> (2,2)
```

El operador `@` realiza el producto matricial. Equivale en estos casos a `np.matmul(A, x)` o `np.matmul(A, B)`. La forma de salida depende de las dimensiones de entrada.

$$A\boldsymbol{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

Diferencia esencial entre * y @

```
A * B  # Multiply matching elements (Hadamard product)
A @ B  # Row-by-column matrix multiplication
```

No son intercambiables. `np.dot()` coincide con `@` en muchos casos simples de 1D y 2D, pero su comportamiento en dimensiones mayores puede diferir. En esta clase utilizaremos `@` para que se reconozca el producto matricial.

6. Arrays columna frente a arrays 1D

```
x = np.array([2, -1])
x_col = np.array([[2], [-1]])

print(x.shape)      # (2,)
print(x_col.shape)  # (2, 1)
```

Las dos representaciones contienen los mismos componentes, pero **no tienen la misma forma**. En EX28 recomendamos vectores 1D para simplificar el código.

7. Imprimir salidas de manera legible

```
print(f"Matrix shape: {A.shape}")
print(f"Transposed matrix:\n{A.T}")
print(f"Result vector: {A @ x}")
```

El salto de línea `\n` permite mostrar la matriz debajo del título. Las unidades deben figurar en las salidas físicas, pero no en los arrays puramente matemáticos.

8. Errores típicos y su interpretación

Situación	Qué revisar
A.shape no coincide con lo esperado	Número de filas y columnas
A @ x genera ValueError	Dimensiones internas incompatibles
A * B da un resultado inesperado	Usar @ si pretendías un producto matricial
A.T parece igual que A	¿Es simétrica o estás usando un vector 1D?

Para EX28: basta con NumPy, un archivo .py y comprobaciones manuales. No necesitamos JSON, pytest, módulos propios ni resolución inversa de sistemas.